

Capture the Flag 3: Data-Driven UI

Goal: Separate application data from the UI and handle multiple pieces of data in a usable interface.

Instructor Note

These activities require your own problem-solving and research skills.

DO NOT use AI to generate the code for you.

You may use AI to:

- Research what a Dart/Flutter feature does
- Understand syntax
- **Explain** an error (not **fix** your error)
- Clarify documentation

You may **NOT** ask AI to generate the solution/code for a flag.

Flag 0 — Create Your Feature Branch

Objective: Create a feature branch before modifying your application.

Create:

`feature/data-ui`

Screenshot:

- Terminal showing your current branch

```
MINGW64/c/Users/admin/my_first_flutter_app
* feature/widgets
master

Shairha@NE013 MINGW64 ~/my_first_flutter_app (feature/widgets)
$ git status
On branch feature/widgets
Your branch is up to date with 'origin/feature/widgets'.

nothing to commit, working tree clean

Shairha@NE013 MINGW64 ~/my_first_flutter_app (feature/widgets)
$ git checkout -b feature/data-ui
Switched to a new branch 'feature/data-ui'

Shairha@NE013 MINGW64 ~/my_first_flutter_app (feature/data-ui)
$ git push origin feature/data-ui
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'feature/data-ui' on GitHub by visiting:
remote:   https://github.com/Shayy511/IT314AB_Ejares/pull/new/feature/data-u
remote:
To https://github.com/Shayy511/IT314AB_Ejares.git
 * [new branch]      feature/data-ui -> feature/data-ui
```

Flag 1 — Identify the Data

Objective: Identify the appropriate Dart data type for the information in your profile.

Research basic Dart data types and identify at least **four different types** that could be used in your application.

Examples:

String
int
double
bool

Create a table identifying the information and its appropriate data type.

Screenshot:

- Your identified data
- The corresponding data types

INFO DATA	DATA TYPES
Name	String
Age	Int
Is Student	Bool
Student status	Double

🚩 Flag 2 — Create Your Data

Objective: Create appropriately typed variables for your profile.

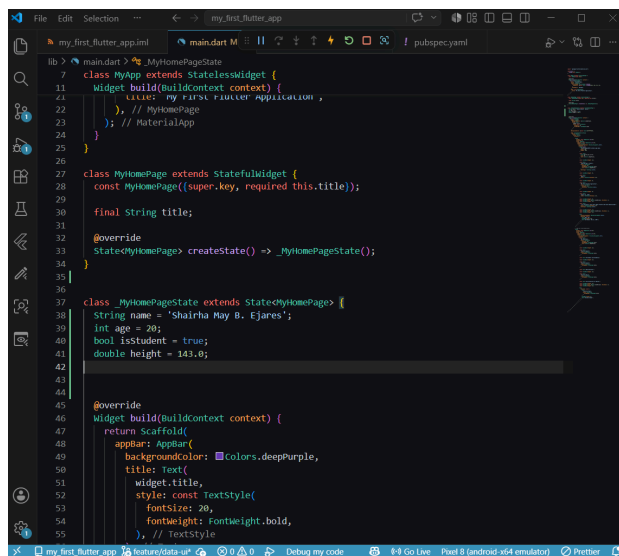
Create variables for at least:

- Name
- Course & Section
- Age
- Hobby
- Height
- Student status

Your data should exist **separately from your UI widgets**.

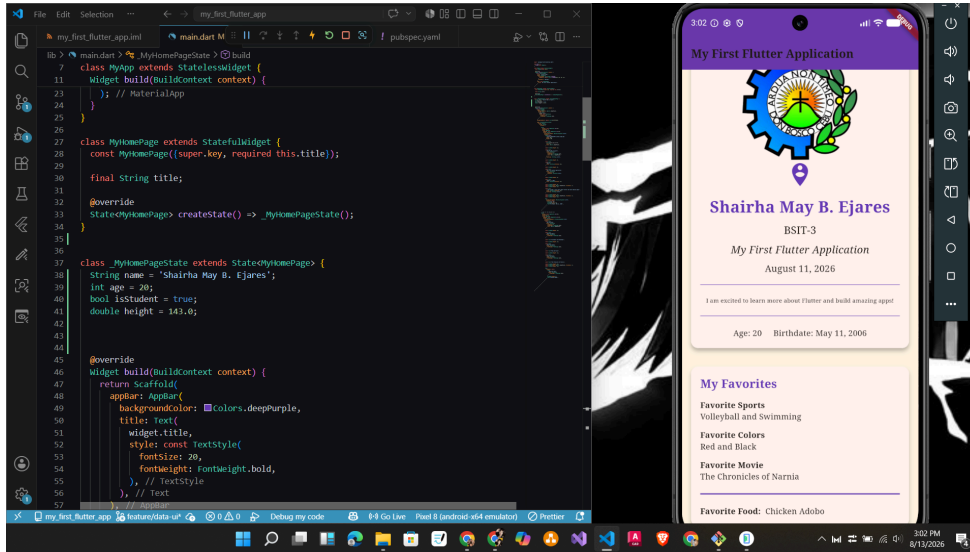
Screenshot:

- Your variables
- Their data types



```
lib > main.dart > % MyHomePageState
7 class MyApp extends StatelessWidget {
11   Widget build(BuildContext context) {
22     // MyHomePage
23     // MaterialApp
24   }
25 }
26
27 class MyHomePage extends StatefulWidget {
28   const MyHomePage({super.key, required this.title});
29
30   final String title;
31
32   @override
33   State<MyHomePage> createState() => _MyHomePageState();
34 }
35
36 class _MyHomePageState extends State<MyHomePage> {
37   String name = 'Shairha May B. Ejares';
38   int age = 20;
39   bool isStudent = true;
40   double height = 143.0;
41
42
43
44
45   @override
46   Widget build(BuildContext context) {
47     return Scaffold(
48       appBar: AppBar(
49         backgroundColor: Colors.deepPurple,
50         title: Text(
51           widget.title,
52           style: TextStyle(
53             fontSize: 20,
54             fontWeight: FontWeight.bold,
55           ), // TextStyle
```

- Application still running



🚩 Flag 3 — Connect Data to UI

Objective: Replace your hardcoded information with your variables.

Your profile should now display information using the variables you created.

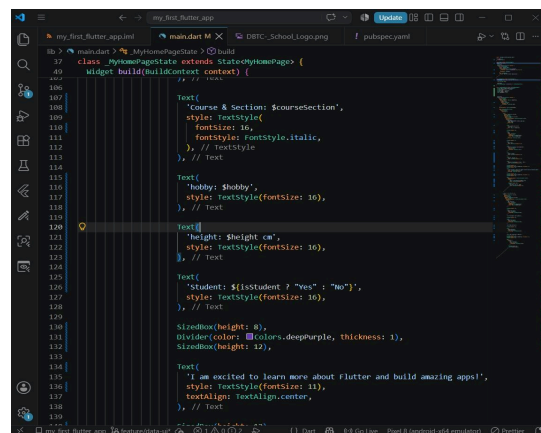
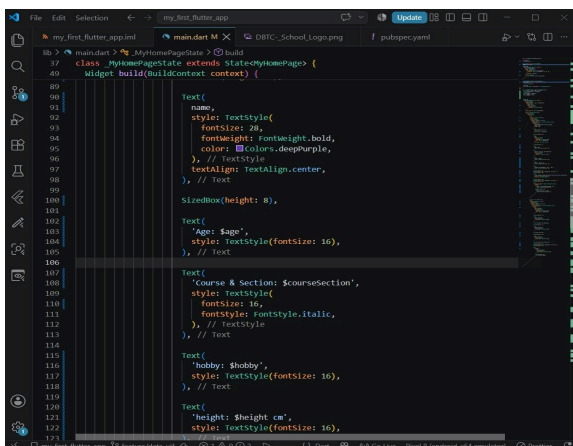
Test:

Change the value of one of your variables.

The UI should automatically display the new value.

Screenshot:

- Variables
- Widgets displaying the variables



```

lib > main.dart
class MyHomePage extends StatefulWidget {
  MyHomePage({super.key, required this.title});

  final String title;

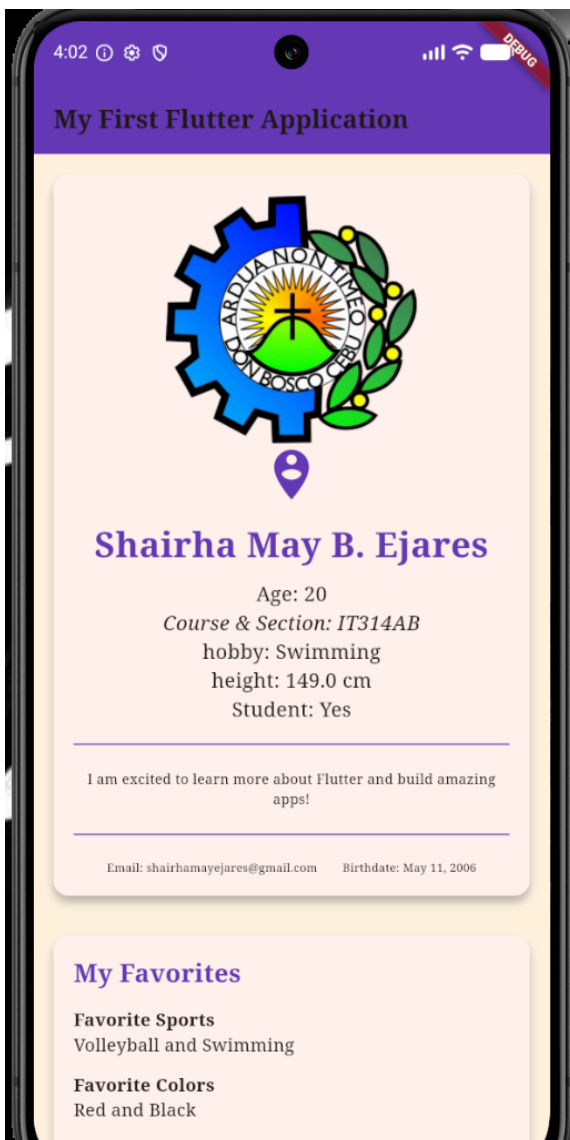
  @override
  State<MyHomePage> createState() => _MyHomePageState();
}

class _MyHomePageState extends State<MyHomePage> {
  String name = "Shairha May B. Ejares";
  String courseSection = "IT314AB";
  int age = 20;
  String hobby = "Swimming";
  double height = 149.0;
  bool isStudent = true;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        backgroundColor: Colors.deepPurple,
        title: Text(
          widget.title,
          style: TextStyle(
            fontSize: 20,
            fontWeight: FontWeight.bold,
          ), // textStyle
        ), // text
      ), // appBar
    );
  }
}

```

- Updated application



Objective: Make your profile image part of your data.

Create a variable containing the path to your profile image.

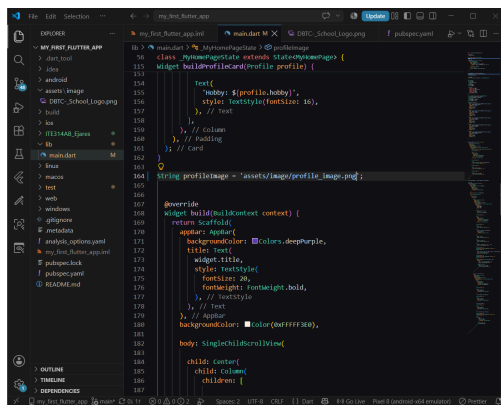
Use the variable when displaying the image.

Then change the variable to another registered image.

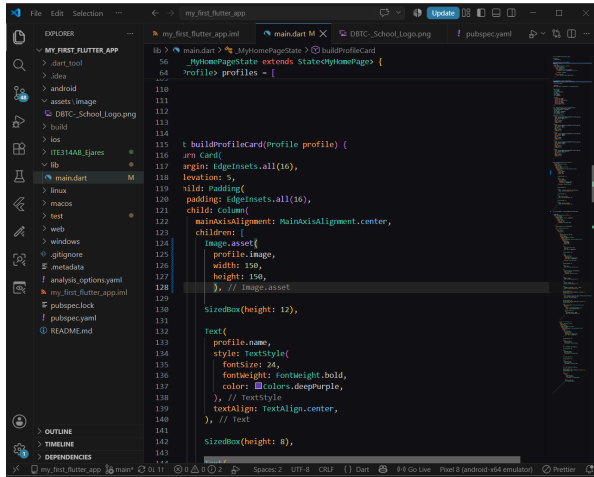
The image displayed by the application should change.

Screenshot:

- Image variable



- Image widget using the variable



- Updated application

Flag 5 — Five Profiles, One Design

Objective: Create at least **five different profile cards** using the same visual design.

Each profile must contain at least:

- Image
- Name
- Course & Section
- Age
- Hobby

The five profiles must contain different information.

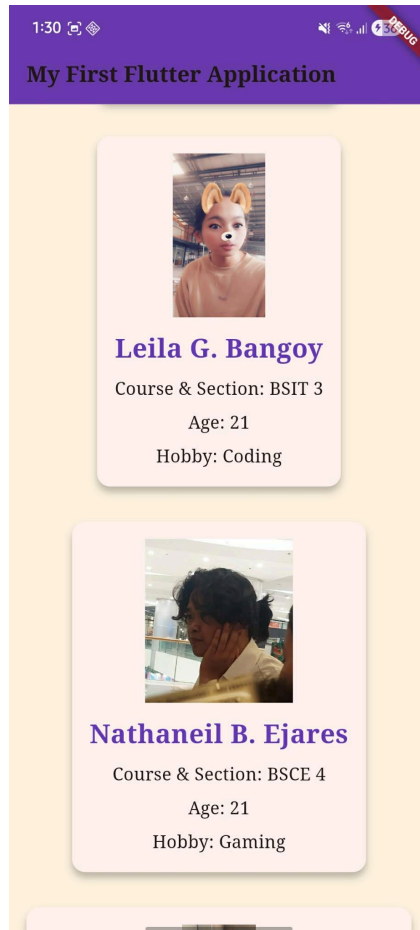
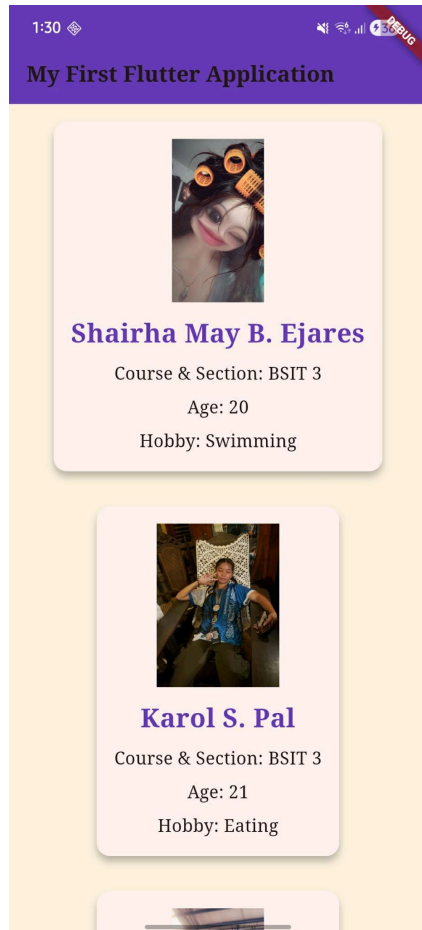
Important

All five profiles should use the **same visual design**.

You are demonstrating that the same UI can represent different data.

Screenshot:

- Your 5 different profile data





Flag 6 — THE HIDDEN PROBLEM

You didn't see your UI?

Objective: Make all five profiles accessible to the user.

You just added five profile cards.

Now run your application.

Something is wrong.

Look carefully at your emulator.

Can you see all five profiles?

Maybe you can see the first one...

Maybe part of the second...

But where are the rest?

That's right.

You can't see them.

Your UI is larger than the available screen.

The profiles exist.

The data exists.

The widgets exist.

But the user can't access everything.

Your Mission:

Figure out how Flutter allows users to **scroll through content that is larger than the available screen**.

Research Flutter's **scrolling widgets** and determine which one is appropriate for your current layout.

You must make it possible to scroll through all five profiles.

Requirement:

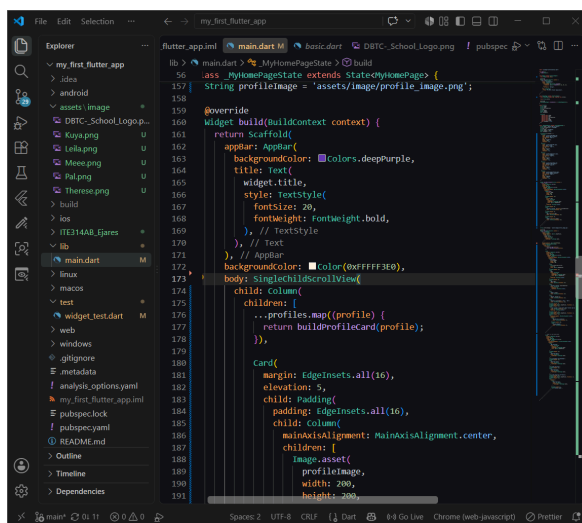
The user must be able to:

Scroll → see Profile 1 → Profile 2 → Profile 3 → Profile 4 → Profile 5

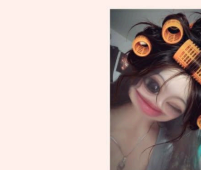
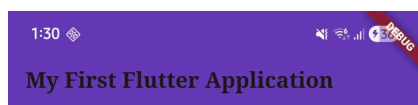
without changing the size of the screen.

Screenshot:

- The scrolling widget in your code



- Top portion of the application

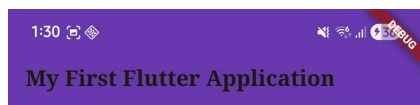


Shairha May B. Ejares

Course & Section: BSIT 3

Age: 20

Hobby: Swimming

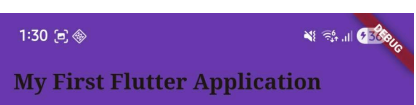


Leila G. Bangoy

Course & Section: BSIT 3

Age: 21

Hobby: Coding



Nathaneil B. Ejares

Course & Section: BSCE 4

Age: 21

Hobby: Gaming



Karol S. Pal

Course & Section: BSIT 3



Nathaneil B. Ejares



**Therese Deanne
Lumactud**

- The rest of the portion of the application showing the later profiles

Flag 7 — Survive the Database

Objective: Prepare your UI for incomplete data.

Now imagine your profiles aren't manually created.

Someday, the data will come from a database.

And real data isn't always perfect.

Some information may be:

- Missing
- Empty
- `null`
- Not provided

Create at least **three profiles with missing information**.

For example:

Profile A

Name: Maria

Course: BSIT

Hobby: **missing**

Profile B

Name: Juan

Course: **missing**

Hobby: Gaming

Profile C

Name: **missing**

Course: MMA

Hobby: Drawing

Your application must **not crash** when information is missing.

Instead, provide an appropriate fallback.

Examples:

Hobby: Not provided

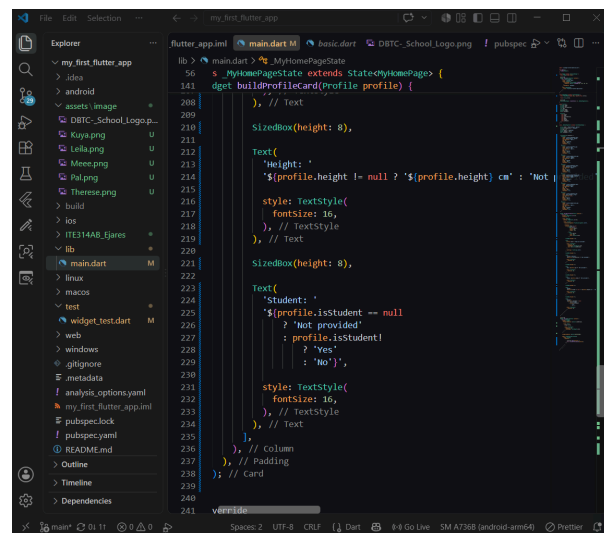
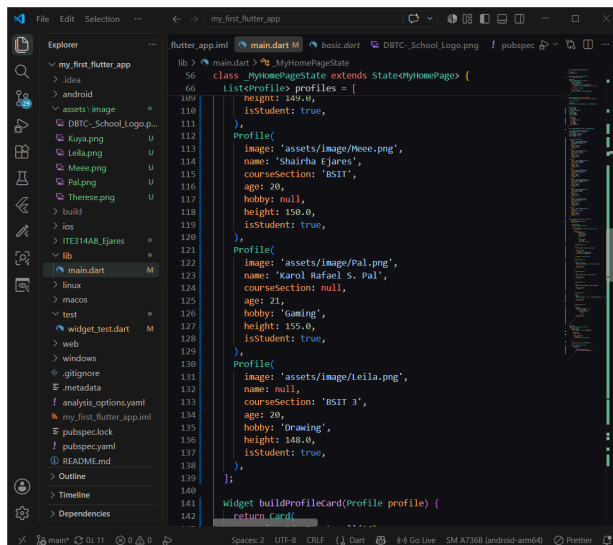
Course: Unknown

Challenge

Your placeholders should look intentional and clean.

Screenshot:

- Incomplete profile data



- Application displaying fallback values
- Application still functioning correctly



Flag 8 — Save Your Progress

Objective: Commit and push your completed Data-Driven UI CTF.

Use an appropriate commit message following proper Git standards.

Branch:

`feature/data-ui`

Screenshot:

- Successful commit
- Current branch
- Terminal showing the commit

